Computer Vision Lab Mid

Name: Muhammad Hammad

Reg No: FA23-BAI-003

Question no 1 : Data setup

```
# ---------- PERSONAL SETTINGS (FA23-BAI-003) ----------
SEED      = 3            # last two digits "03"
HOG_CELL  = (16, 16)     # last digit 3 is odd
SIFT_GRID = 4            # last digit 3 is in 0-4, so 4x4 = 16 keypoints

# ---------- FIXED SETTINGS ----------
CLASS_IDS   = [1, 5, 6, 7]
CLASS_NAMES = {1: "eosinophil", 5: "monocyte", 6: "neutrophil", 7: "platelet"}
N_PER_CLASS = {"train": 100, "val": 30, "test": 30}
IMG_SIZE    = 128
```

```
!pip install medmnist opencv-python scikit-learn scikit-image matplotlib

import numpy as np
import cv2
import matplotlib.pyplot as plt
from medmnist import BloodMNIST
```

```
Collecting medmnist
  Downloading medmnist-3.0.2-py3-none-any.whl.metadata (14 kB)
Requirement already satisfied: opencv-python in /usr/local/lib/python3.13/dist-packages (5.0.0.93)
Requirement already satisfied: scikit-learn in /usr/local/lib/python3.13/dist-packages (1.6.1)
Requirement already satisfied: scikit-image in /usr/local/lib/python3.13/dist-packages (0.25.2)
Requirement already satisfied: matplotlib in /usr/local/lib/python3.13/dist-packages (3.10.0)
Requirement already satisfied: numpy in /usr/local/lib/python3.13/dist-packages (from medmnist)
(2.1.3)
Requirement already satisfied: pandas in /usr/local/lib/python3.13/dist-packages (from medmnist)
(2.2.3)
Requirement already satisfied: tqdm in /usr/local/lib/python3.13/dist-packages (from medmnist)
(4.67.3)
Requirement already satisfied: Pillow in /usr/local/lib/python3.13/dist-packages (from medmnist)
(11.3.0)
Collecting fire (from medmnist)
  Downloading fire-0.7.1-py3-none-any.whl.metadata (5.8 kB)
Requirement already satisfied: torch in /usr/local/lib/python3.13/dist-packages (from medmnist)
(2.11.0+cpu)
Requirement already satisfied: torchvision in /usr/local/lib/python3.13/dist-packages (from medmnist)
(0.26.0+cpu)
Requirement already satisfied: scipy>=1.6.0 in /usr/local/lib/python3.13/dist-packages (from scikit-
learn) (1.16.3)
Requirement already satisfied: joblib>=1.2.0 in /usr/local/lib/python3.13/dist-packages (from scikit-
learn) (1.6.0)
Requirement already satisfied: threadpoolctl>=3.1.0 in /usr/local/lib/python3.13/dist-packages (from
scikit-learn) (3.7.0)
Requirement already satisfied: networkx>=3.0 in /usr/local/lib/python3.13/dist-packages (from scikit-
image) (3.7)
Requirement already satisfied: imageio!=2.35.0,>=2.33 in /usr/local/lib/python3.13/dist-packages
(from scikit-image) (2.38.1)
Requirement already satisfied: tifffile>=2022.8.12 in /usr/local/lib/python3.13/dist-packages (from
scikit-image) (2026.10.15)
Requirement already satisfied: packaging>=21 in /usr/local/lib/python3.13/dist-packages (from scikit-
image) (26.3)
Requirement already satisfied: lazy-loader>=0.4 in /usr/local/lib/python3.13/dist-packages (from
scikit-image) (0.6)
Requirement already satisfied: contourpy>=1.0.1 in /usr/local/lib/python3.13/dist-packages (from
matplotlib) (1.4.0)
```

```
Requirement already satisfied: cycler>=0.10 in /usr/local/lib/python3.13/dist-packages (from
matplotlib) (0.12.1)
Requirement already satisfied: fonttools>=4.22.0 in /usr/local/lib/python3.13/dist-packages (from
matplotlib) (4.67.0)
Requirement already satisfied: kiwisolver>=1.3.1 in /usr/local/lib/python3.13/dist-packages (from
matplotlib) (1.5.1)
Requirement already satisfied: pyparsing>=2.3.1 in /usr/local/lib/python3.13/dist-packages (from
matplotlib) (3.3.3)
Requirement already satisfied: python-dateutil>=2.7 in /usr/local/lib/python3.13/dist-packages (from
matplotlib) (2.9.0.post0)
Requirement already satisfied: cloudpickle>=3.0 in /usr/local/lib/python3.13/dist-packages (from
joblib>=1.2.0->scikit-learn) (3.1.2)
Requirement already satisfied: six>=1.5 in /usr/local/lib/python3.13/dist-packages (from python-
dateutil>=2.7->matplotlib) (1.17.0)
Requirement already satisfied: termcolor in /usr/local/lib/python3.13/dist-packages (from fire-
>medmnist) (3.3.0)
Requirement already satisfied: pytz>=2020.1 in /usr/local/lib/python3.13/dist-packages (from pandas-
>medmnist) (2025.2)
Requirement already satisfied: tzdata>=2022.7 in /usr/local/lib/python3.13/dist-packages (from
pandas->medmnist) (2026.5)
Requirement already satisfied: filelock in /usr/local/lib/python3.13/dist-packages (from torch-
>medmnist) (4.2.4)
Requirement already satisfied: typing-extensions>=4.10.0 in /usr/local/lib/python3.13/dist-packages
(from torch->medmnist) (4.16.0)
Requirement already satisfied: setuptools<82 in /usr/local/lib/python3.13/dist-packages (from torch-
>medmnist) (80.10.2)
Requirement already satisfied: sympy>=1.13.3 in /usr/local/lib/python3.13/dist-packages (from torch-
>medmnist) (1.14.0)
Requirement already satisfied: jinja2 in /usr/local/lib/python3.13/dist-packages (from torch-
>medmnist) (3.1.6)
Requirement already satisfied: fsspec>=0.8.5 in /usr/local/lib/python3.13/dist-packages (from torch-
>medmnist) (2025.12.0)
Requirement already satisfied: mpmath<1.4,>=1.1.0 in /usr/local/lib/python3.13/dist-packages (from
sympy>=1.13.3->torch->medmnist) (1.3.0)
Requirement already satisfied: MarkupSafe>=2.0 in /usr/local/lib/python3.13/dist-packages (from
jinja2->torch->medmnist) (3.0.4)
Downloading medmnist-3.0.2-py3-none-any.whl (25 kB)
Downloading fire-0.7.1-py3-none-any.whl (115 kB)
Installing collected packages: fire, medmnist
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 2/2 [medmnist]
Successfully installed fire-0.7.1 medmnist-3.0.2
```

```python
rng = np.random.default_rng(SEED)   # the seeded random generator

def prepare(img_rgb):
    """RGB 64x64 -> grayscale 128x128"""
    gray = cv2.cvtColor(img_rgb, cv2.COLOR_RGB2GRAY)
    return cv2.resize(gray, (IMG_SIZE, IMG_SIZE))

def build_split(split_name, n_per_class):
    ds = BloodMNIST(split=split_name, download=True, size=64)
    imgs, labels = ds.imgs, ds.labels.ravel()
    X, y = [], []
    for c in CLASS_IDS:
        idx = np.where(labels == c)[0]                       # all images of class c
        pick = rng.choice(idx, size=n_per_class, replace=False)  # random pick with seed
        for i in pick:
            X.append(prepare(imgs[i]))
            y.append(c)
    return np.array(X), np.array(y)

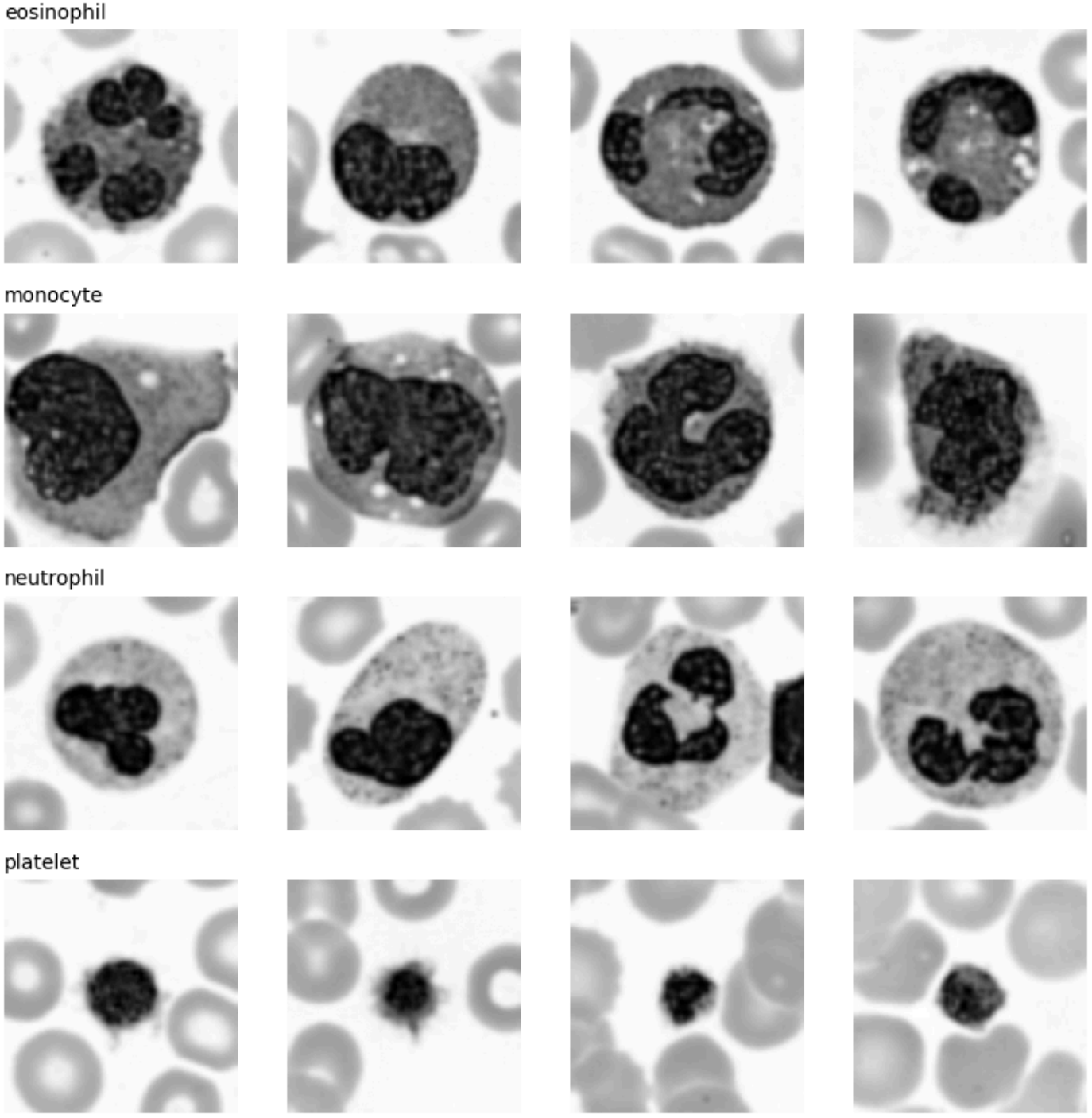
X_train, y_train = build_split("train", N_PER_CLASS["train"])
X_val,   y_val   = build_split("val",   N_PER_CLASS["val"])
X_test,  y_test  = build_split("test",  N_PER_CLASS["test"])
```

```
100%|██████████| 156M/156M [00:37<00:00, 4.12MB/s]
```

```
for name, X, y in [("train", X_train, y_train),
                   ("val",   X_val,   y_val),
                   ("test",  X_test,  y_test)]:
    print(f"{name:5s} X shape: {X.shape}  y shape: {y.shape}")
    counts = {CLASS_NAMES[c]: int((y == c).sum()) for c in CLASS_IDS}
    print("      class counts:", counts)
```

```
train X shape: (400, 128, 128)  y shape: (400,)
      class counts: {'eosinophil': 100, 'monocyte': 100, 'neutrophil': 100, 'platelet': 100}
val   X shape: (120, 128, 128)  y shape: (120,)
      class counts: {'eosinophil': 30, 'monocyte': 30, 'neutrophil': 30, 'platelet': 30}
test  X shape: (120, 128, 128)  y shape: (120,)
      class counts: {'eosinophil': 30, 'monocyte': 30, 'neutrophil': 30, 'platelet': 30}
```

```
fig, axes = plt.subplots(4, 4, figsize=(8, 8))
for r, c in enumerate(CLASS_IDS):
    imgs_c = X_train[y_train == c][:4]
    for k in range(4):
        axes[r, k].imshow(imgs_c[k], cmap="gray")
        axes[r, k].axis("off")
    axes[r, 0].set_title(CLASS_NAMES[c], loc="left", fontsize=10)
plt.tight_layout()
plt.show()
```



Question no 2HOG + SVM

```
from skimage.feature import hog

def extract_hog(img):
    """one 128x128 gray image (0-255) -> HOG feature vector"""
    img01 = img.astype(np.float32) / 255.0      # scale to 0-1 (same scale as the noise test later)
    return hog(img01,
               orientations=9,
               pixels_per_cell=HOG_CELL,        # (16,16) from your settings
               cells_per_block=(2, 2),
               block_norm="L2-Hys")

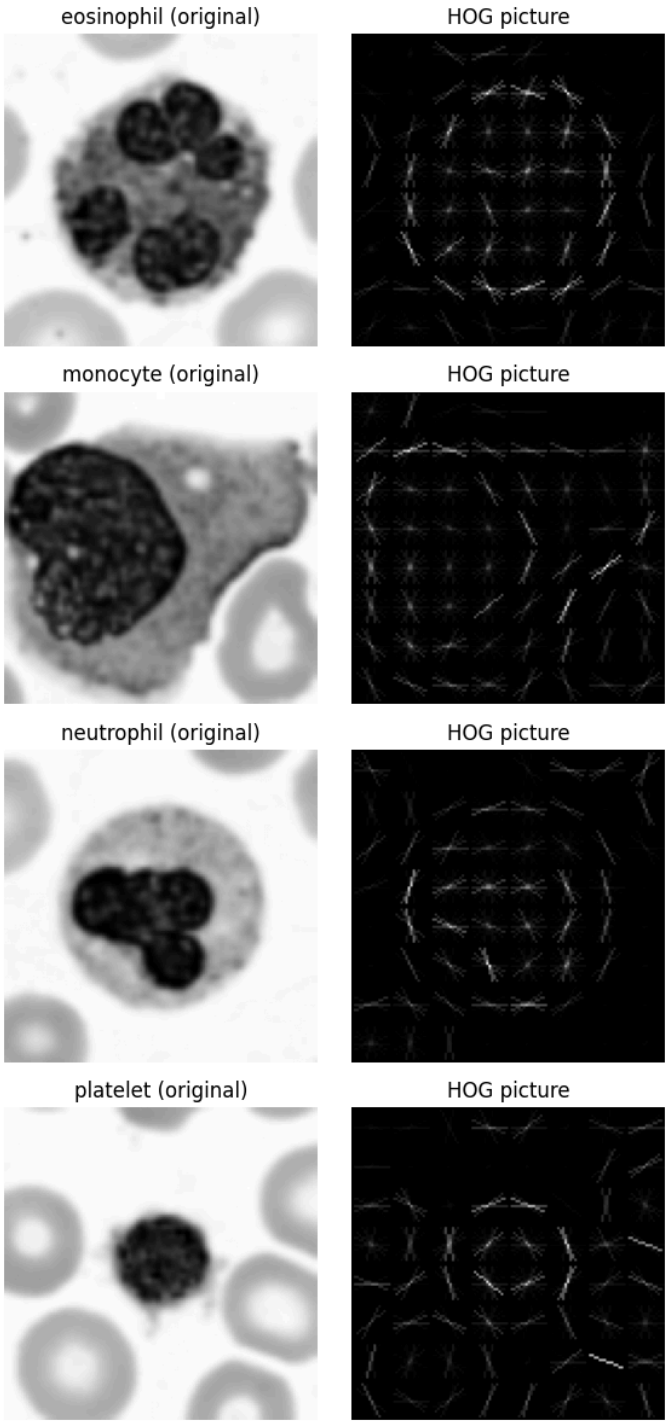
def hog_features(images):
    return np.array([extract_hog(im) for im in images])

Xtr_hog = hog_features(X_train)
Xva_hog = hog_features(X_val)
```

```
print("Train HOG shape:", Xtr_hog.shape)
print("Val HOG shape:  ", Xva_hog.shape)
print("HOG feature length:", Xtr_hog.shape[1])
```

```
Train HOG shape: (400, 1764)
Val HOG shape:   (120, 1764)
HOG feature length: 1764
```

```
fig, axes = plt.subplots(4, 2, figsize=(6, 12))
for r, c in enumerate(CLASS_IDS):
    img = X_train[y_train == c][0]
    _, hog_img = hog(img.astype(np.float32) / 255.0,
                     orientations=9, pixels_per_cell=HOG_CELL,
                     cells_per_block=(2, 2), block_norm="L2-Hys",
                     visualize=True)
    axes[r, 0].imshow(img, cmap="gray")
    axes[r, 0].set_title(CLASS_NAMES[c] + " (original)")
    axes[r, 1].imshow(hog_img, cmap="gray")
    axes[r, 1].set_title("HOG picture")
    axes[r, 0].axis("off"); axes[r, 1].axis("off")
plt.tight_layout(); plt.show()
```




```
from sklearn.svm import LinearSVC
from sklearn.metrics import accuracy_score, f1_score, confusion_matrix, ConfusionMatrixDisplay

best_acc, best_C, hog_model = -1, None, None
for C in [0.1, 1, 10]:
    clf = LinearSVC(C=C, max_iter=10000, random_state=SEED)
    clf.fit(Xtr_hog, y_train)
    acc = accuracy_score(y_val, clf.predict(Xva_hog))
    print(f"C = {C:<4}  validation accuracy = {acc:.4f}")
    if acc > best_acc:
        best_acc, best_C, hog_model = acc, C, clf

print("\nBest C:", best_C)
```

```
C = 0.1   validation accuracy = 0.7417
C = 1     validation accuracy = 0.6750
C = 10    validation accuracy = 0.6833
```

```
Best C: 0.1
```

```
val_pred = hog_model.predict(Xva_hog)
print("Validation accuracy:", round(accuracy_score(y_val, val_pred), 4))
print("Validation macro-F1:", round(f1_score(y_val, val_pred, average="macro"), 4))

cm = confusion_matrix(y_val, val_pred, labels=CLASS_IDS)
print(cm)
ConfusionMatrixDisplay(cm, display_labels=[CLASS_NAMES[c] for c in CLASS_IDS]).plot(cmap="Blues")
plt.title("HOG + SVM: validation confusion matrix")
plt.show()
```

```
Validation accuracy: 0.7417
Validation macro-F1: 0.7442
[[17  7  6  0]
 [ 8 22  0  0]
 [ 7  2 21  0]
 [ 0  0  1 29]]
```

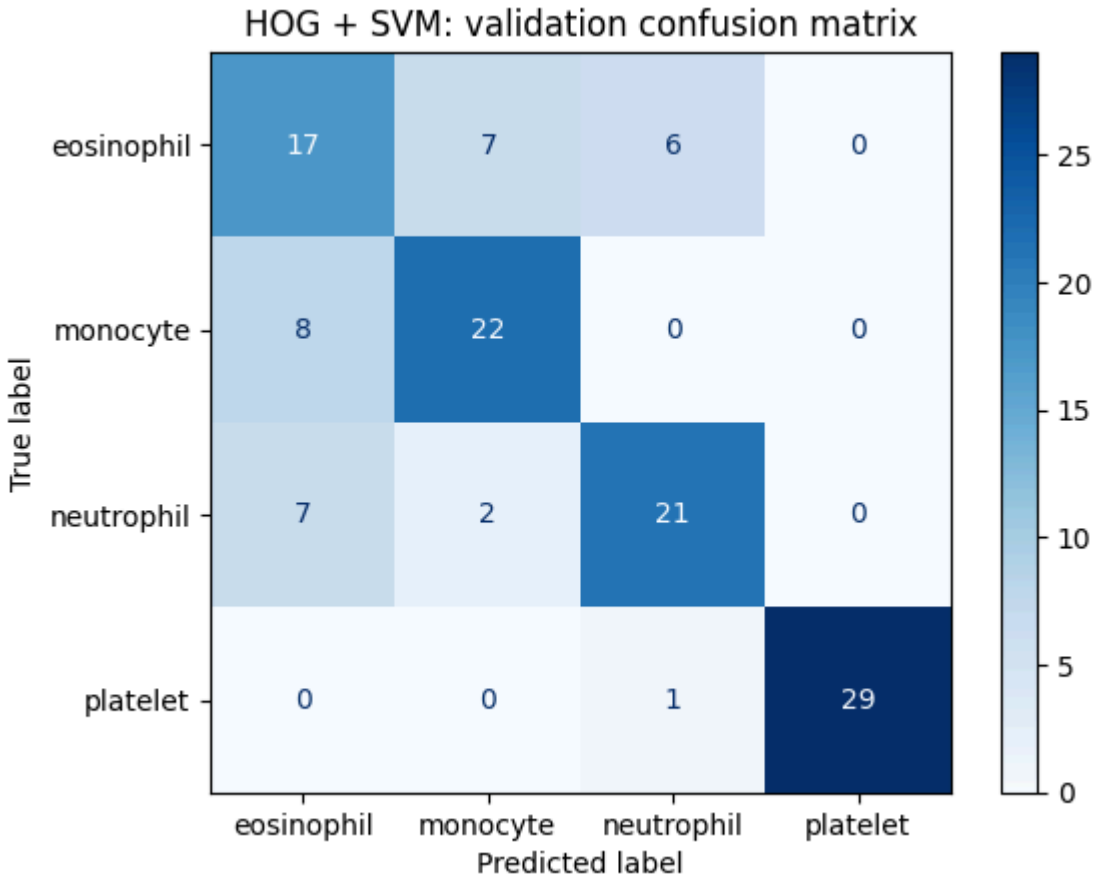


image 128 × 128, cell 16 × 16, block 2 × 2 cells, 9 orientations.

Cells per side: 128 / 16 = 8, so the grid is 8 × 8 cells. Blocks per side (2 × 2 cells, moving 1 cell at a time): 8 - 2 + 1 = 7, so 7 × 7 = 49 blocks. Numbers per block: 2 × 2 cells x 9 orientations = 36. Total: 49 × 36 = 1764.

QUESTION NO 3 :SIFT + SVM

Calculation

Grid points = 4 × 4 = 16

Numbers per descriptor = 128

Feature length = 16 × 128 = 2048

```
sift = cv2.SIFT_create()

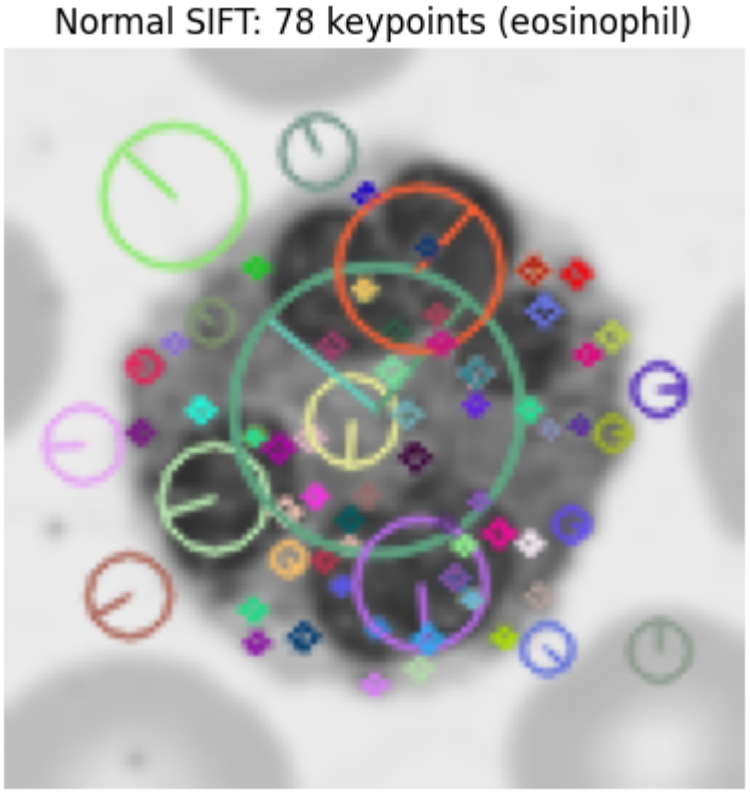
kp_counts = []
for img in X_train:
    kps, desc = sift.detectAndCompute(img, None)   # img must be uint8
    kp_counts.append(len(kps))
kp_counts = np.array(kp_counts)

print("Average keypoints per image:", round(kp_counts.mean(), 2))
print("Minimum keypoints:", kp_counts.min())
print("Maximum keypoints:", kp_counts.max())
print("Images with zero keypoints:", int((kp_counts == 0).sum()))
```

```
Average keypoints per image: 72.21
Minimum keypoints: 6
Maximum keypoints: 263
Images with zero keypoints: 0
```

```
img = X_train[0]
kps, _ = sift.detectAndCompute(img, None)
drawn = cv2.drawKeypoints(img, kps, None, flags=cv2.DRAW_MATCHES_FLAGS_DRAW_RICH_KEYPOINTS)

plt.imshow(drawn, cmap="gray")
plt.title(f"Normal SIFT: {len(kps)} keypoints ({CLASS_NAMES[y_train[0]]})")
plt.axis("off")
plt.show()
```



an SVM needs every image to have a feature vector of the same length, but normal SIFT gives a different number of keypoints, so a different vector length, for each image.

```
# build the grid keypoints once
step = IMG_SIZE / SIFT_GRID                        # 128 / 4 = 32
centers = (np.arange(SIFT_GRID) + 0.5) * step      # 16, 48, 80, 112
print("Grid centres:", centers)

def make_grid_keypoints():
    return [cv2.KeyPoint(float(x), float(y), 32)  # size = 32
            for y in centers for x in centers]

def extract_sift_grid(img):
    kps = make_grid_keypoints()
    kps, desc = sift.compute(img, kps)
    assert desc.shape == (SIFT_GRID * SIFT_GRID, 128), desc.shape
    return desc.flatten()                          # join into one vector

def sift_features(images):
    return np.array([extract_sift_grid(im) for im in images])

Xtr_sift = sift_features(X_train)
Xva_sift = sift_features(X_val)

print("Train SIFT shape:", Xtr_sift.shape)
print("Val SIFT shape:  ", Xva_sift.shape)
print("SIFT feature length:", Xtr_sift.shape[1])
```

```
Grid centres: [ 16.  48.  80. 112.]
Train SIFT shape: (400, 2048)
Val SIFT shape:   (120, 2048)
SIFT feature length: 2048
```

```
best_acc_s, best_C_s, sift_model = -1, None, None
for C in [0.1, 1, 10]:
    clf = LinearSVC(C=C, max_iter=10000, random_state=SEED)
    clf.fit(Xtr_sift, y_train)
    acc = accuracy_score(y_val, clf.predict(Xva_sift))
```

```
    print(f"C = {C:<4}  validation accuracy = {acc:.4f}")
    if acc > best_acc_s:
        best_acc_s, best_C_s, sift_model = acc, C, clf

print("\nBest C:", best_C_s)
```

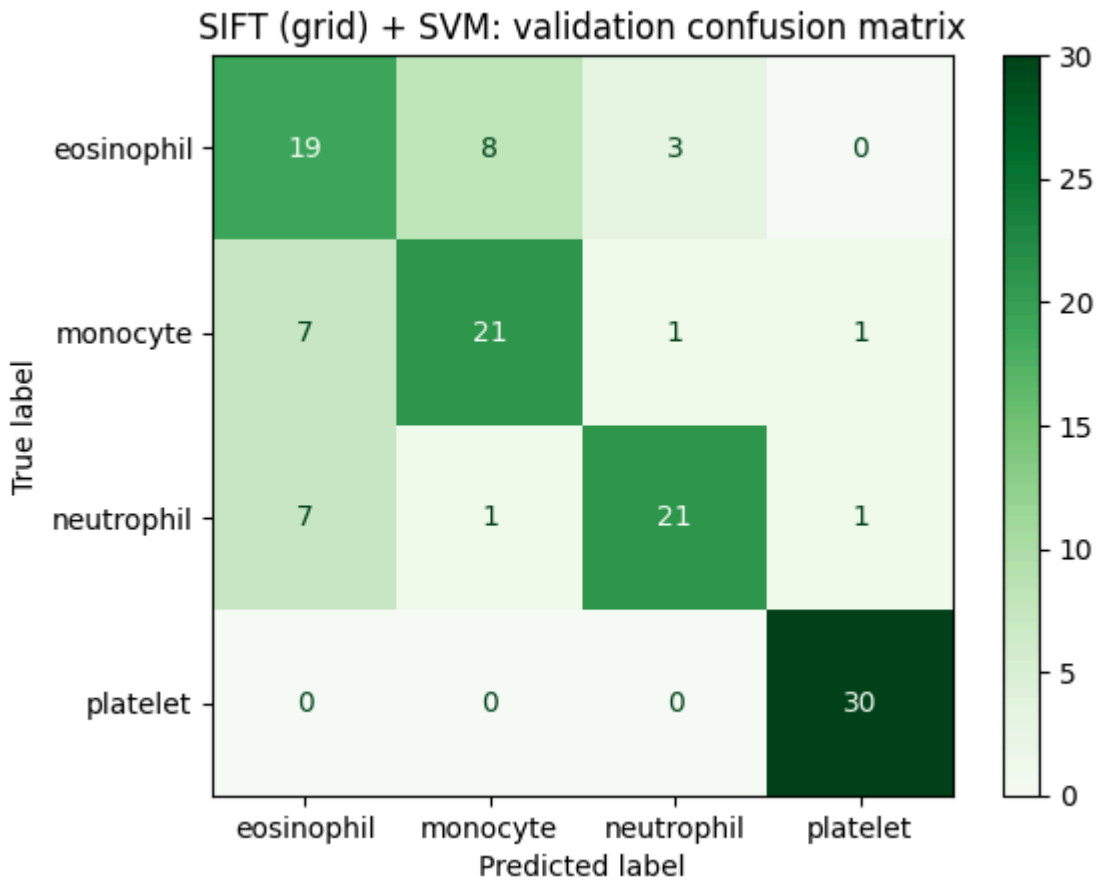
```
/usr/local/lib/python3.13/dist-packages/sklearn/svm/_base.py:1249: ConvergenceWarning: Liblinear
failed to converge, increase the number of iterations.
  warnings.warn(
C = 0.1   validation accuracy = 0.7583
/usr/local/lib/python3.13/dist-packages/sklearn/svm/_base.py:1249: ConvergenceWarning: Liblinear
failed to converge, increase the number of iterations.
  warnings.warn(
C = 1     validation accuracy = 0.7583
C = 10    validation accuracy = 0.7583

Best C: 0.1
/usr/local/lib/python3.13/dist-packages/sklearn/svm/_base.py:1249: ConvergenceWarning: Liblinear
failed to converge, increase the number of iterations.
  warnings.warn(
```

```
val_pred_s = sift_model.predict(Xva_sift)
print("Validation accuracy:", round(accuracy_score(y_val, val_pred_s), 4))
print("Validation macro-F1:", round(f1_score(y_val, val_pred_s, average="macro"), 4))

cm_s = confusion_matrix(y_val, val_pred_s, labels=CLASS_IDS)
print(cm_s)
ConfusionMatrixDisplay(cm_s, display_labels=[CLASS_NAMES[c] for c in CLASS_IDS]).plot(cmap="Greens")
plt.title("SIFT (grid) + SVM: validation confusion matrix")
plt.show()
```

```
Validation accuracy: 0.7583
Validation macro-F1: 0.7586
[[19  8  3  0]
 [ 7 21  1  1]
 [ 7  1 21  1]
 [ 0  0  0 30]]
```



QUESTION NO 4 : Robustness

I think HOG + SVM will lose more accuracy under rotation because it uses many small cells, so turning the picture moves edge directions between cells, while SIFT's larger smoothed patches change less. I think HOG + SVM will also lose more under noise because it measures gradients on raw pixels, so random grain creates fake edges, while SIFT smooths the image first.

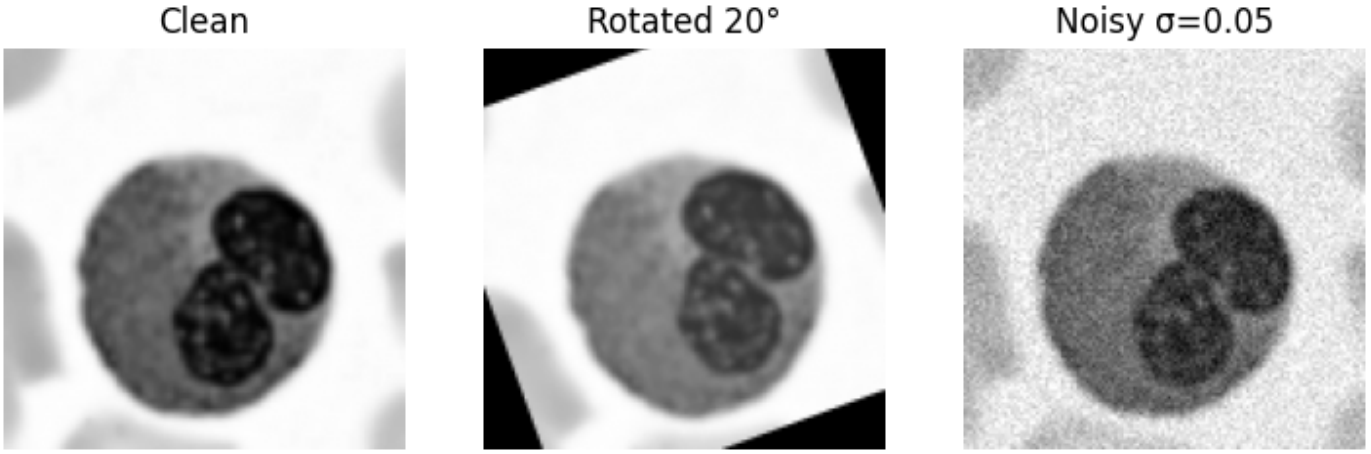
```
def rotate_img(img, angle=20):
    h, w = img.shape
    M = cv2.getRotationMatrix2D((w / 2, h / 2), angle, 1.0)   # turn around the centre
    return cv2.warpAffine(img, M, (w, h), borderValue=0)      # black corners

def add_noise(img, sigma=0.05, rng=None):
    img01 = img.astype(np.float32) / 255.0                    # scale to 0-1
    noisy = img01 + rng.normal(0, sigma, img01.shape)         # add Gaussian grain
    noisy = np.clip(noisy, 0, 1)                              # keep inside 0-1
    return (noisy * 255).astype(np.uint8)                     # back to uint8 for HOG/SIFT code

noise_rng = np.random.default_rng(SEED)   # seeded, so your noise is repeatable

X_test_rot   = np.array([rotate_img(im, 20) for im in X_test])
X_test_noisy = np.array([add_noise(im, 0.05, noise_rng) for im in X_test])
```

```
fig, axes = plt.subplots(1, 3, figsize=(9, 3))
for ax, im, t in zip(axes, [X_test[0], X_test_rot[0], X_test_noisy[0]],
                     ["Clean", "Rotated 20°", "Noisy σ=0.05"]):
    ax.imshow(im, cmap="gray"); ax.set_title(t); ax.axis("off")
plt.show()
```




```
def evaluate(model, feat_fn, images, labels):
    X_feat = np.array([feat_fn(im) for im in images])
    return accuracy_score(labels, model.predict(X_feat))

cases = {"Clean": X_test, "Rotated": X_test_rot, "Noisy": X_test_noisy}

results = {"HOG + SVM": {}, "SIFT + SVM": {}}
for case, imgs in cases.items():
    results["HOG + SVM"][case]  = evaluate(hog_model,  extract_hog,       imgs, y_test)
    results["SIFT + SVM"][case] = evaluate(sift_model, extract_sift_grid, imgs, y_test)

print(f"{'Model':<12}{'Clean':>10}{'Rotated':>10}{'Noisy':>10}")
for m, r in results.items():
    print(f"{m:<12}{r['Clean']:>10.4f}{r['Rotated']:>10.4f}{r['Noisy']:>10.4f}")
```

```
Model            Clean   Rotated     Noisy
HOG + SVM       0.7583    0.6583    0.5083
SIFT + SVM      0.7917    0.4833    0.8083
```

```
for m, r in results.items():
    print(f"{m}: drop under rotation = {r['Clean'] - r['Rotated']:.4f}, "
          f"drop under noise = {r['Clean'] - r['Noisy']:.4f}")
```

```
HOG + SVM: drop under rotation = 0.1000, drop under noise = 0.2500
SIFT + SVM: drop under rotation = 0.3083, drop under noise = -0.0167
```

RESULTS OF THE CODE

Model Clean Rotated Noisy

HOG + SVM 0.7583 0.6583 0.5083

SIFT + SVM 0.7917 0.4833 0.8083

QUESTION NO 5

```
def top_confusions(cm, name):
    pairs = []
    for i in range(len(CLASS_IDS)):
        for j in range(i + 1, len(CLASS_IDS)):
            total = cm[i, j] + cm[j, i]      # mistakes in both directions
            pairs.append((total, CLASS_NAMES[CLASS_IDS[i]], CLASS_NAMES[CLASS_IDS[j]], cm[i, j],
cm[j, i]))
    pairs.sort(reverse=True)
    print(name)
    for total, a, b, ab, ba in pairs[:3]:
```

```
        print(f"  {a} <-> {b}: {total} mistakes ({a} predicted as {b}: {ab}, {b} predicted as {a}:
{ba})")

top_confusions(cm,   "HOG + SVM (validation)")
top_confusions(cm_s, "SIFT + SVM (validation)")
```

```
HOG + SVM (validation)
  eosinophil <-> monocyte: 15 mistakes (eosinophil predicted as monocyte: 7, monocyte predicted as
eosinophil: 8)
  eosinophil <-> neutrophil: 13 mistakes (eosinophil predicted as neutrophil: 6, neutrophil predicted
as eosinophil: 7)
  monocyte <-> neutrophil: 2 mistakes (monocyte predicted as neutrophil: 0, neutrophil predicted as
monocyte: 2)
SIFT + SVM (validation)
  eosinophil <-> monocyte: 15 mistakes (eosinophil predicted as monocyte: 8, monocyte predicted as
eosinophil: 7)
  eosinophil <-> neutrophil: 10 mistakes (eosinophil predicted as neutrophil: 3, neutrophil predicted
as eosinophil: 7)
  monocyte <-> neutrophil: 2 mistakes (monocyte predicted as neutrophil: 1, neutrophil predicted as
monocyte: 1)
```

Question 5: Short answers and AI log

Part 1: Short answers

(a) Why does HOG use block normalisation?

Gradient strength depends on lighting, stain intensity and contrast, so the same cell shape gives large gradients on a dark slide and small ones on a pale slide. Block normalisation divides each 2×2-cell block by its own L2 norm (with L2-Hys the values are also clipped at 0.2 and normalised again). The feature then describes the *pattern of edge directions* rather than how bright or contrasty the image is. Because the blocks overlap, each cell is normalised in several neighbourhoods, so the 1764-length vector is more stable under local shadows and uneven staining.

(b) Why is it wrong to choose C using the test set?

The test set is meant to give an unbiased estimate of how the model does on images it has never seen. If C is picked because it scores best on the test set, the test data has influenced the model (data leakage), so the reported test accuracy is optimistically biased. With only 120 test images, the "best" C could just be the luckiest one. That is why C was chosen on the validation set (C = 0.1 for both models) and the test set was used only once, for the final results.

Part 2: Most-confused classes

Both models confused **eosinophil ↔ monocyte** most, with **15 validation mistakes each**:

Model	Eosinophil → Monocyte	Monocyte → Eosinophil	Total	Eosinophils correct
HOG + SVM	7	8	15	17 / 30
SIFT + SVM	8	7	15	19 / 30

Eosinophil was the weakest class in both models; it was also confused with neutrophil (13 HOG, 10 SIFT mistakes). Platelets were almost perfect (29/30 and 30/30) because their small size and shape are very different from the other cells.

Why grayscale makes this harder: in a stained blood smear, the main cue for an eosinophil is its bright **orange-red (eosin) granules**, while a monocyte has **pale blue-grey cytoplasm**. Both are

similar-sized cells with a large purple nucleus. Converting RGB to grayscale maps orange-red and blue-grey to similar grey levels, so the colour difference disappears and HOG/SIFT only see edges and texture, which look alike for the two cells. Upscaling from 64×64 to 128×128 also blurs the fine granule texture and the nucleus shape (two-lobed vs kidney-shaped) that could otherwise separate them.

Part 3: AI log

#	Prompt (one line)	What I changed	How I checked the result was correct
1	"Load BloodMNIST, keep classes 1, 5, 6, 7 and pick 100/30/30 images per class with a seed"	Used my SEED = 3, converted to grayscale and resized to 128×128, sampled with `replace=False`	Printed shapes (400,128,128) / (120,128,128) / (120,128,128) and class counts 100 / 30 / 30; plotted a 4×4 grid of sample images
2	"Extract HOG features with skimage and tune LinearSVC C on the validation set"	Set my cell size (16,16), scaled pixels to 0–1, tried C in [0.1, 1, 10] on validation only	Printed feature length 1764 and matched my hand calculation 7×7 blocks × 36 = 1764; viewed the HOG pictures for each class
3	"How do I get a fixed-length SIFT vector for an SVM?"	Replaced detected keypoints with my 4×4 grid (centres 16, 48, 80, 112, size 32) and added an `assert` on the descriptor shape	Printed descriptor shape (16, 128) and feature length 2048 = 16 × 128; drew normal SIFT keypoints on an image
4	"Write a 20° rotation and Gaussian noise (σ = 0.05) function for the robustness test"	Seeded the noise generator, clipped values to 0–1, converted back to uint8	Displayed the clean, rotated and noisy versions of the same test image side by side
5	"Help me write the Q5 short answers in a text cell"	Rewrote the answers in my own words and used my own confusion-matrix numbers	Checked the numbers against the `top_confusions` output above (eosinophil ↔ monocyte = 15 for both models)